UML 建模作业

姓名：陆伊琳　　学号：2503408056　　日期：10.30

一、微信小程序「茶颜悦色」——修改用户名 / 密码的事件流

1. 事件流描述

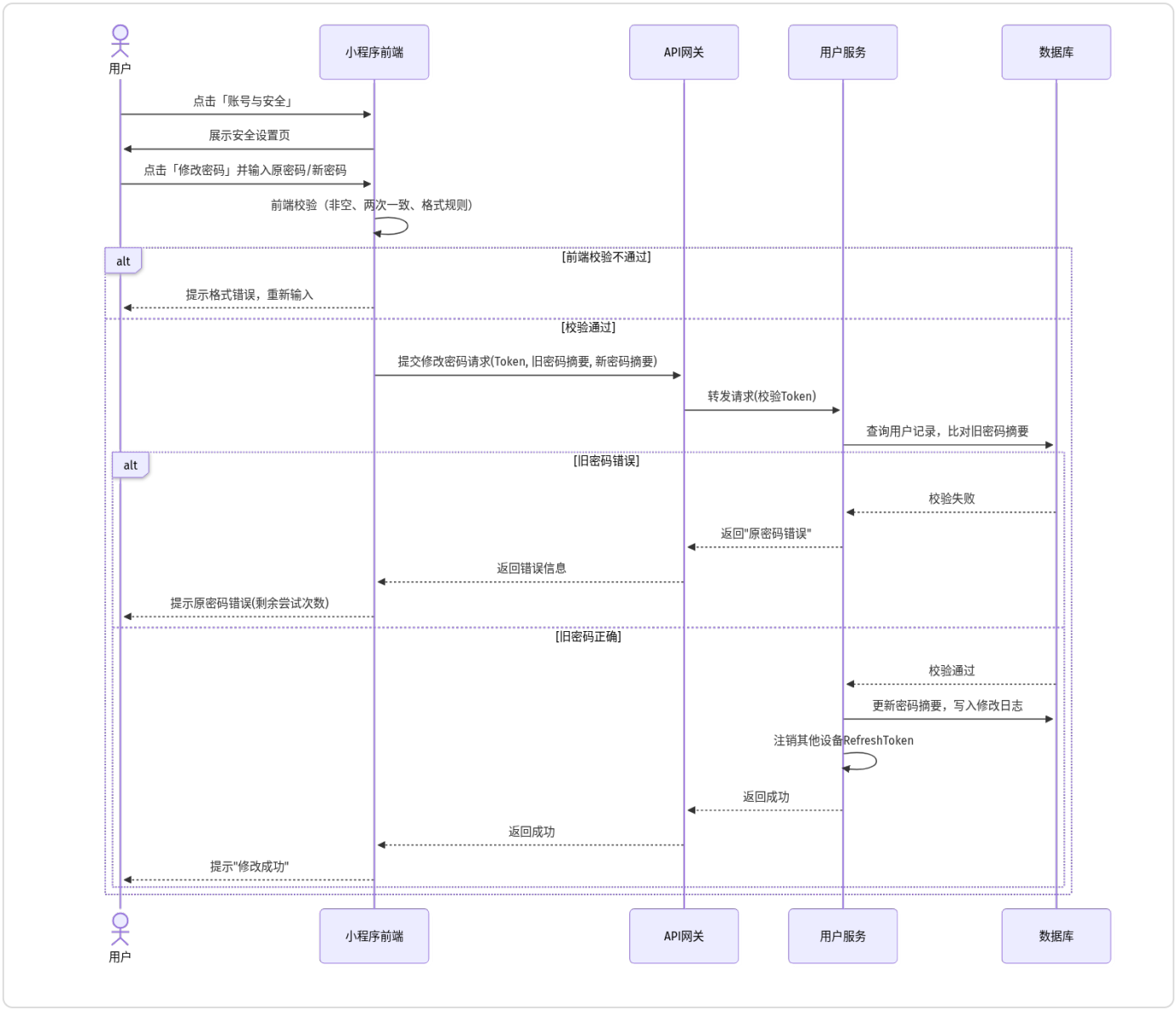
基本事件流（以修改密码为例）

1. 用户打开茶颜悦色小程序，进入「我的」页面；
2. 用户点击「账号与安全」，系统展示安全设置页；
3. 用户点击「修改密码」，系统弹出身份验证界面；
4. 用户输入原密码与新密码（两次确认），点击提交；
5. 系统校验原密码正确、新密码符合规则（8–20 位，含字母和数字）；
6. 系统更新密码，提示「修改成功」，并使其他设备的登录态失效；
7. 用例结束。

备选事件流

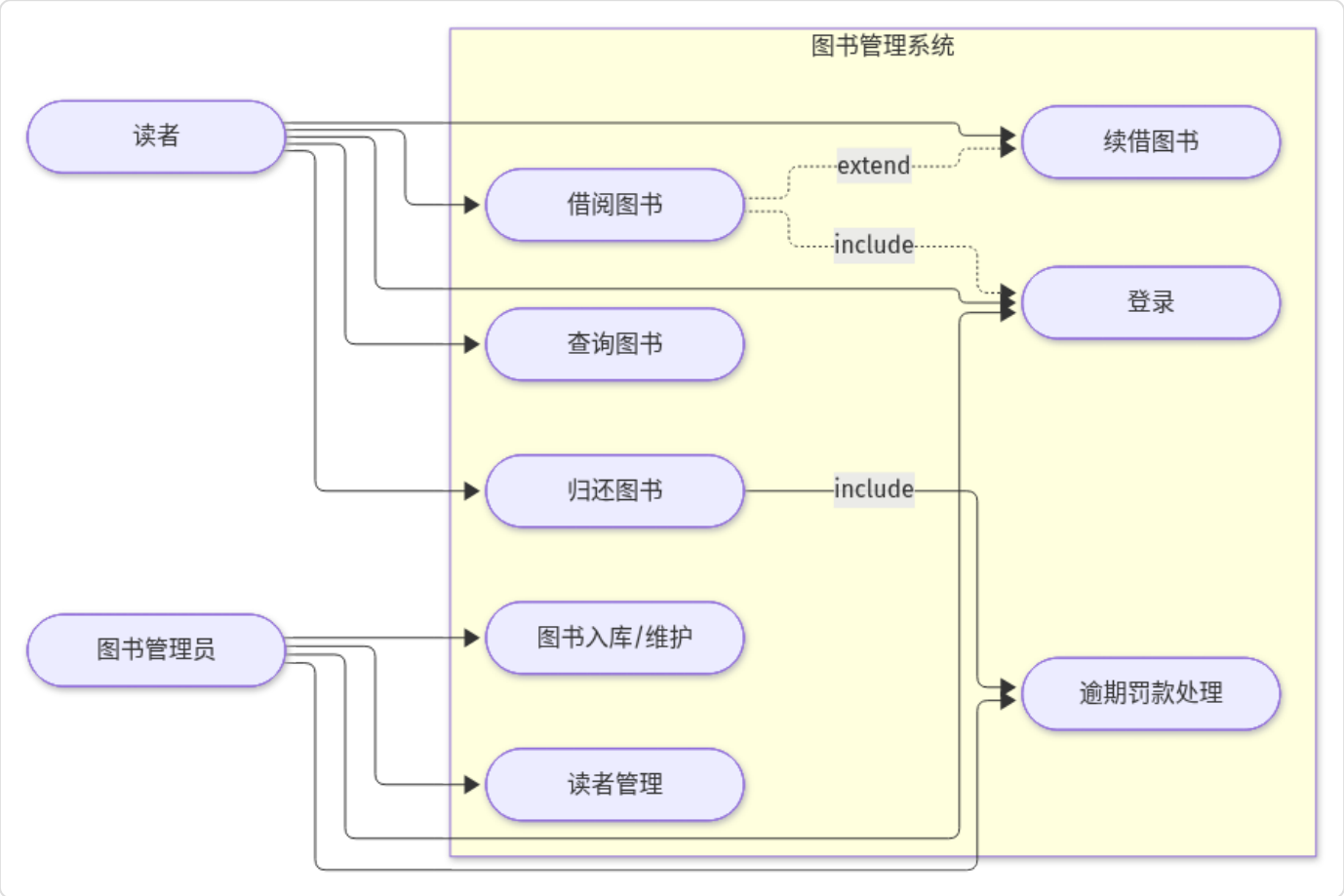
- 3a. 用户忘记原密码 → 跳转「手机验证码找回」分支；
- 5a. 原密码错误 → 提示错误，返回第 4 步（连续 5 次错误锁定 10 分钟）；
- 5b. 新密码不符合规则或两次输入不一致 → 提示原因，返回第 4 步；
- 网络异常 → 提示「网络繁忙，请稍后重试」。

2. 事件流时序图（Sequence Diagram）



二、图书管理系统 UML 图

1. 用例图（Use Case Diagram）

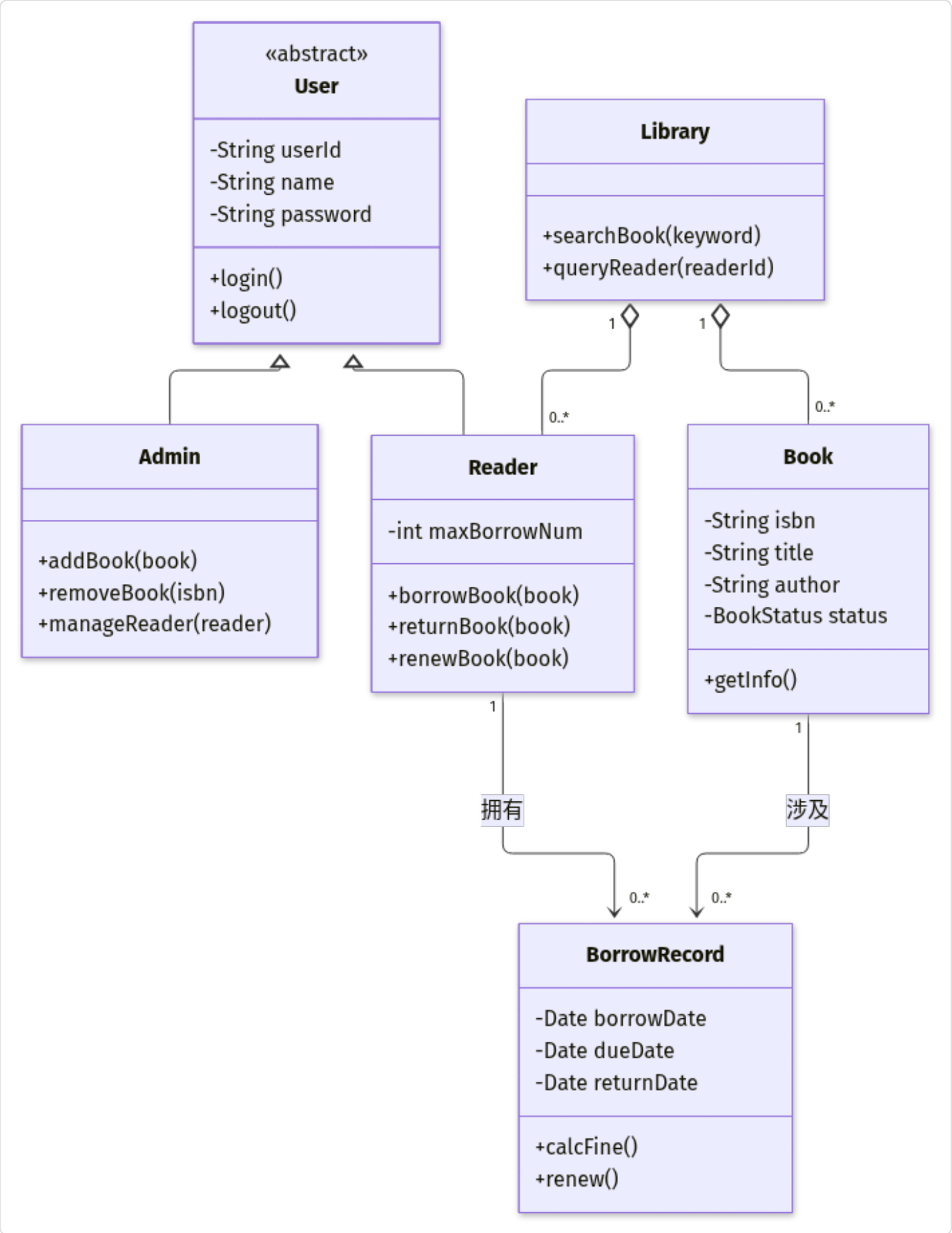



```
graph LR
    subgraph 图书管理系统
        UC1([登录])
        UC2([查询图书])
        UC3([借阅图书])
        UC4([归还图书])
        UC5([续借图书])
        UC6([图书入库/维护])
        UC7([读者管理])
        UC8([逾期罚款处理])
    end
    Reader([读者])
    Admin([图书管理员])

    Reader --> UC1
    Reader --> UC2
    Reader --> UC3
    Reader --> UC4
    Reader --> UC5
    Admin --> UC1
    Admin --> UC6
```

```
    Admin --> UC7
    Admin --> UC8
    UC3 -.->|include| UC1
    UC4 -.->|include| UC8
    UC3 -.->|extend| UC5
```

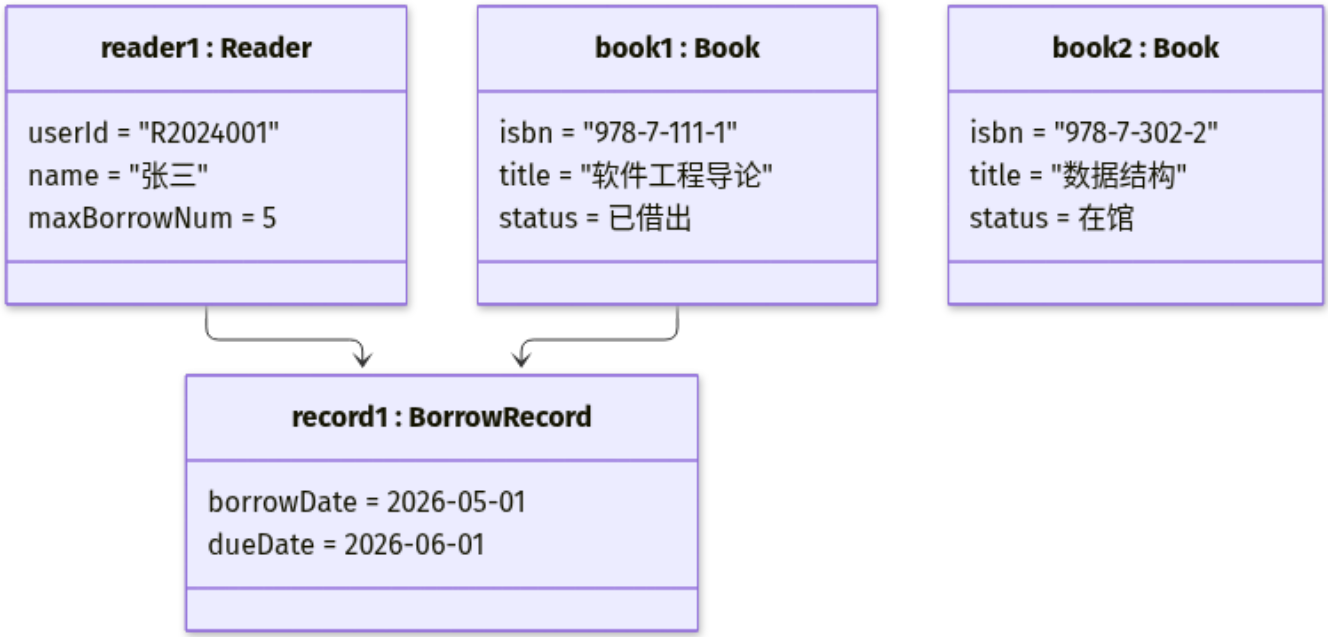
2. 类图（Class Diagram）




```
classDiagram
    class User {
```

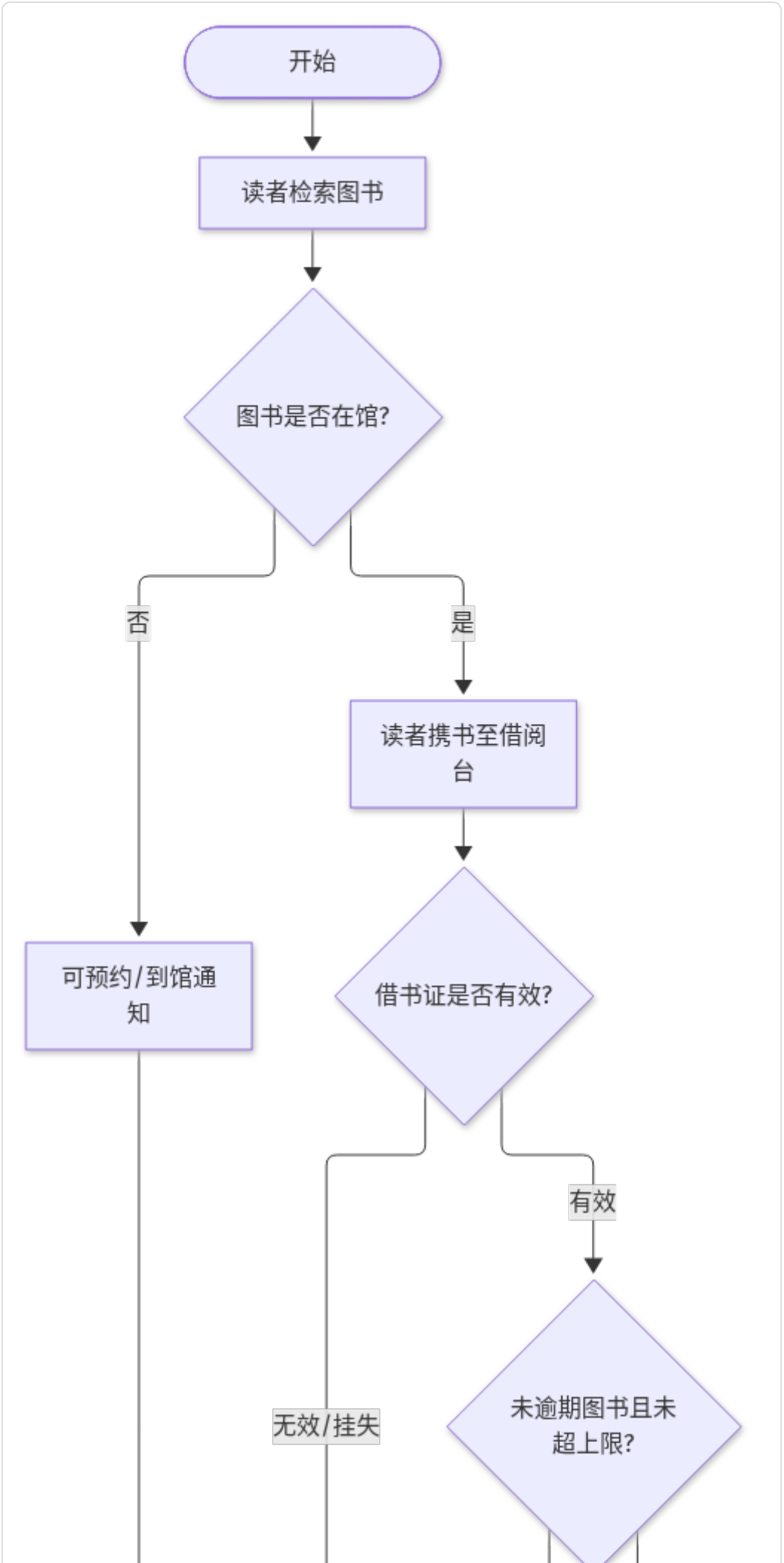
```
    <<abstract>>
    -String userId
    -String name
    -String password
    +login()
    +logout()
}
class Reader {
    -int maxBorrowNum
    +borrowBook(book)
    +returnBook(book)
    +renewBook(book)
}
class Admin {
    +addBook(book)
    +removeBook(isbn)
    +manageReader(reader)
}
class Book {
    -String isbn
    -String title
    -String author
    -BookStatus status
    +getInfo()
}
class BorrowRecord {
    -Date borrowDate
    -Date dueDate
    -Date returnDate
    +calcFine()
    +renew()
}
class Library {
    +searchBook(keyword)
    +queryReader(readerId)
}
User <|-- Reader
User <|-- Admin
Library "1" o-- "0..*" Book
Library "1" o-- "0..*" Reader
Reader "1" --> "0..*" BorrowRecord : 拥有
Book "1" --> "0..*" BorrowRecord : 涉及
```

3. 对象图（Object Diagram，某一时刻的快照）



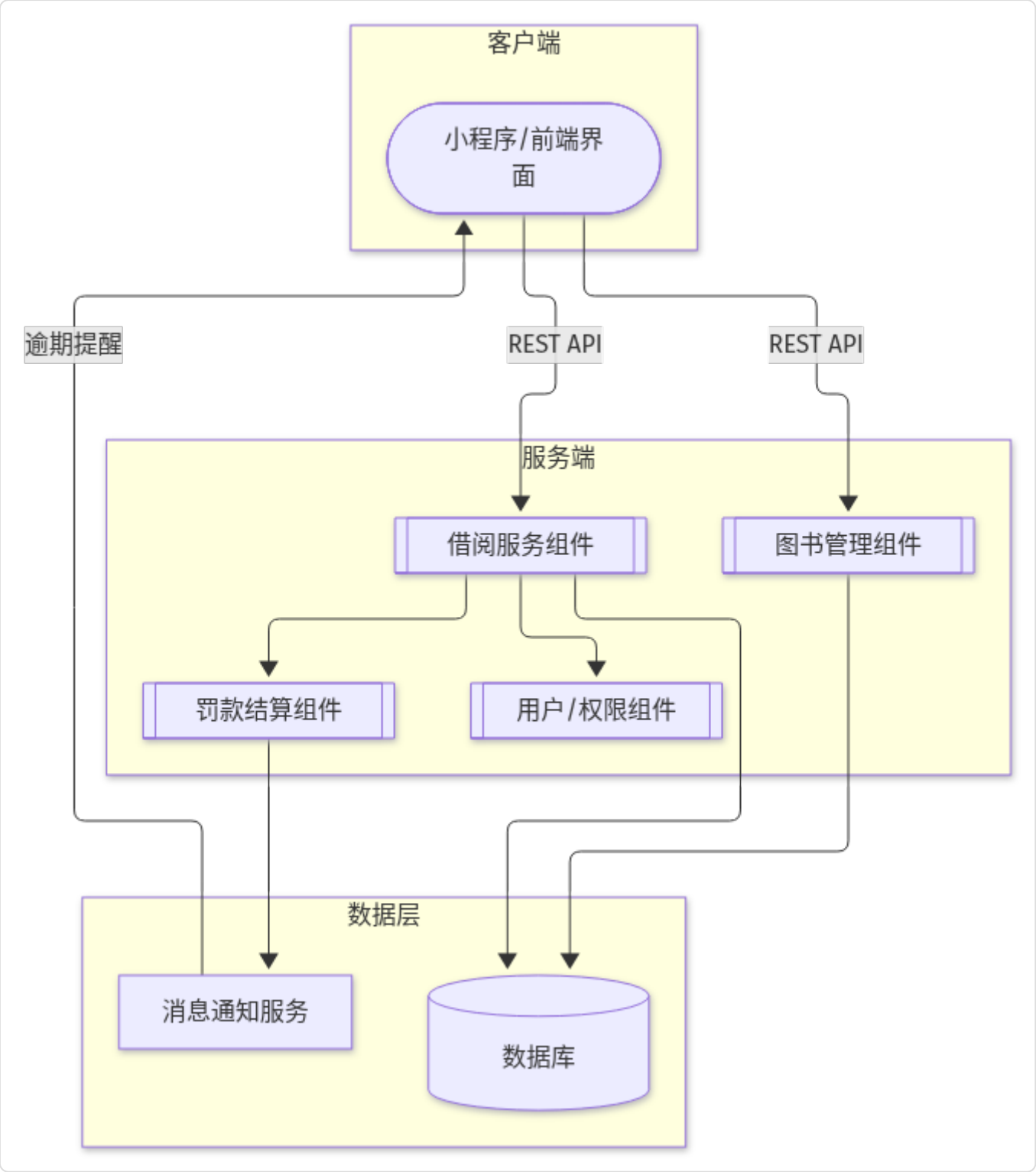

```
classDiagram
    class reader1["reader1 : Reader"] {
        userId = "R2024001"
        name = "张三"
        maxBorrowNum = 5
    }
    class book1["book1 : Book"] {
        isbn = "978-7-111-1"
        title = "软件工程导论"
        status = 已借出
    }
    class book2["book2 : Book"] {
        isbn = "978-7-302-2"
        title = "数据结构"
        status = 在馆
    }
    class record1["record1 : BorrowRecord"] {
        borrowDate = 2026-05-01
        dueDate = 2026-06-01
    }
    reader1 --> record1
    book1 --> record1
```

4. 活动图（Activity Diagram——借书流程）


开始
读者检索图书
图书是否在馆?
否
是
可预约/到馆通知
读者携书至借阅台
借书证是否有效?
无效/挂失
有效
未逾期图书且未超上限?

```
flowchart TD
    A([开始]) --> B[读者检索图书]
    B --> C{图书是否在馆?}
    C -- 否 --> D[可预约/到馆通知] --> Z([结束])
    C -- 是 --> E[读者携书至借阅台]
    E --> F{借书证是否有效?}
    F -- 无效/挂失 --> G[拒绝借阅] --> Z
    F -- 有效 --> H{未逾期图书且未超上限?}
    H -- 否 --> G
    H -- 是 --> I[扫描图书与借书证]
    I --> J[系统写入借阅记录, 更新库存状态]
    J --> K[打印/显示应还日期]
    K --> L[取书离开]
    L --> Z
```

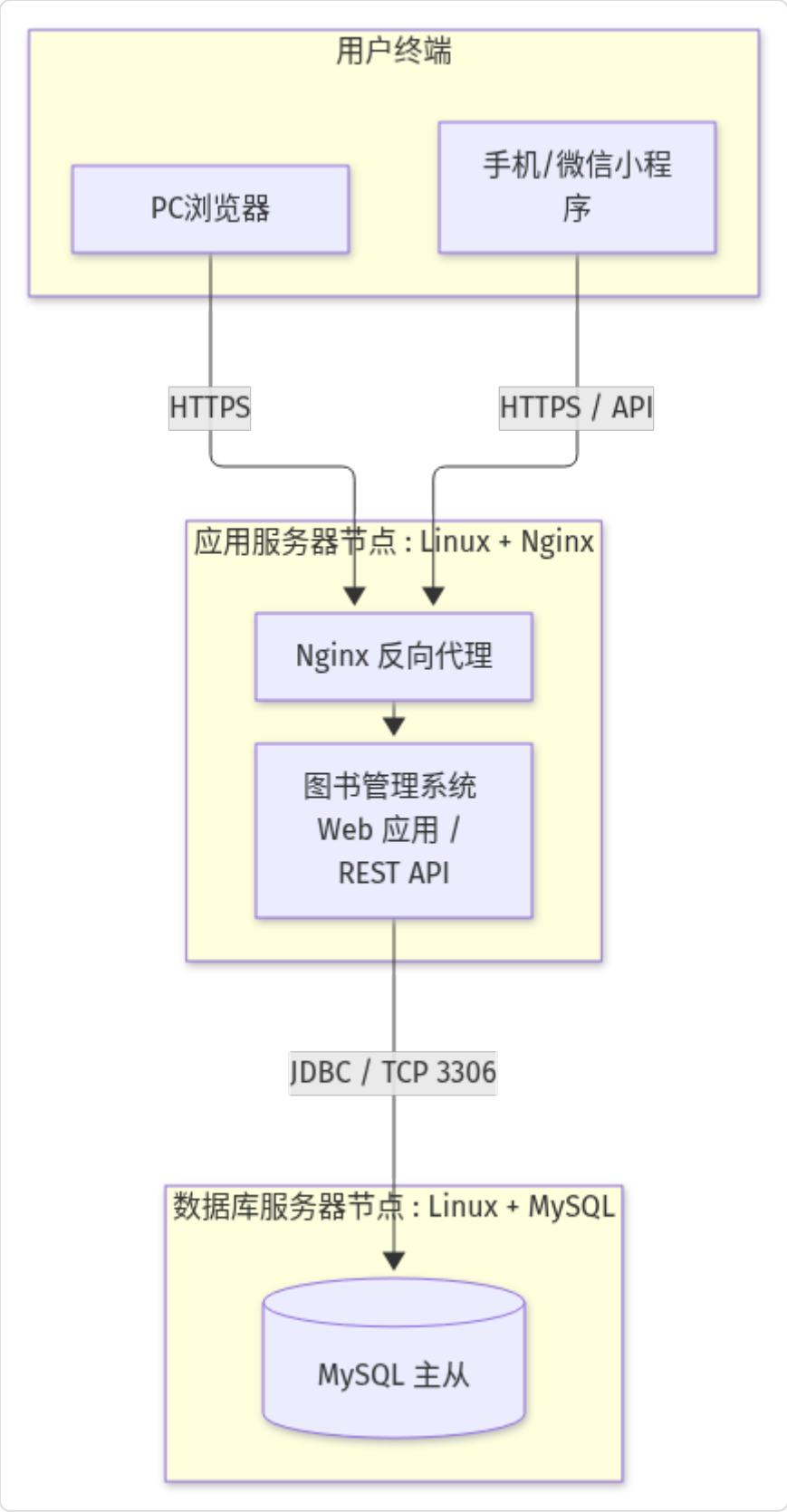
5. 组件图（Component Diagram）




```
graph TB
    subgraph 客户端
        WX([小程序/前端界面])
    end
    subgraph 服务端
        C1[[借阅服务组件]]
        C2[[图书管理组件]]
        C3[[用户/权限组件]]
```

```
    C4[[罚款结算组件]]
end
subgraph 数据层
    DB[(数据库)]
    MSG[消息通知服务]
end
WX -->|REST API| C1
WX -->|REST API| C2
C1 --> C3
C1 --> C4
C1 --> DB
C2 --> DB
C4 --> MSG
MSG -->|逾期提醒| WX
```

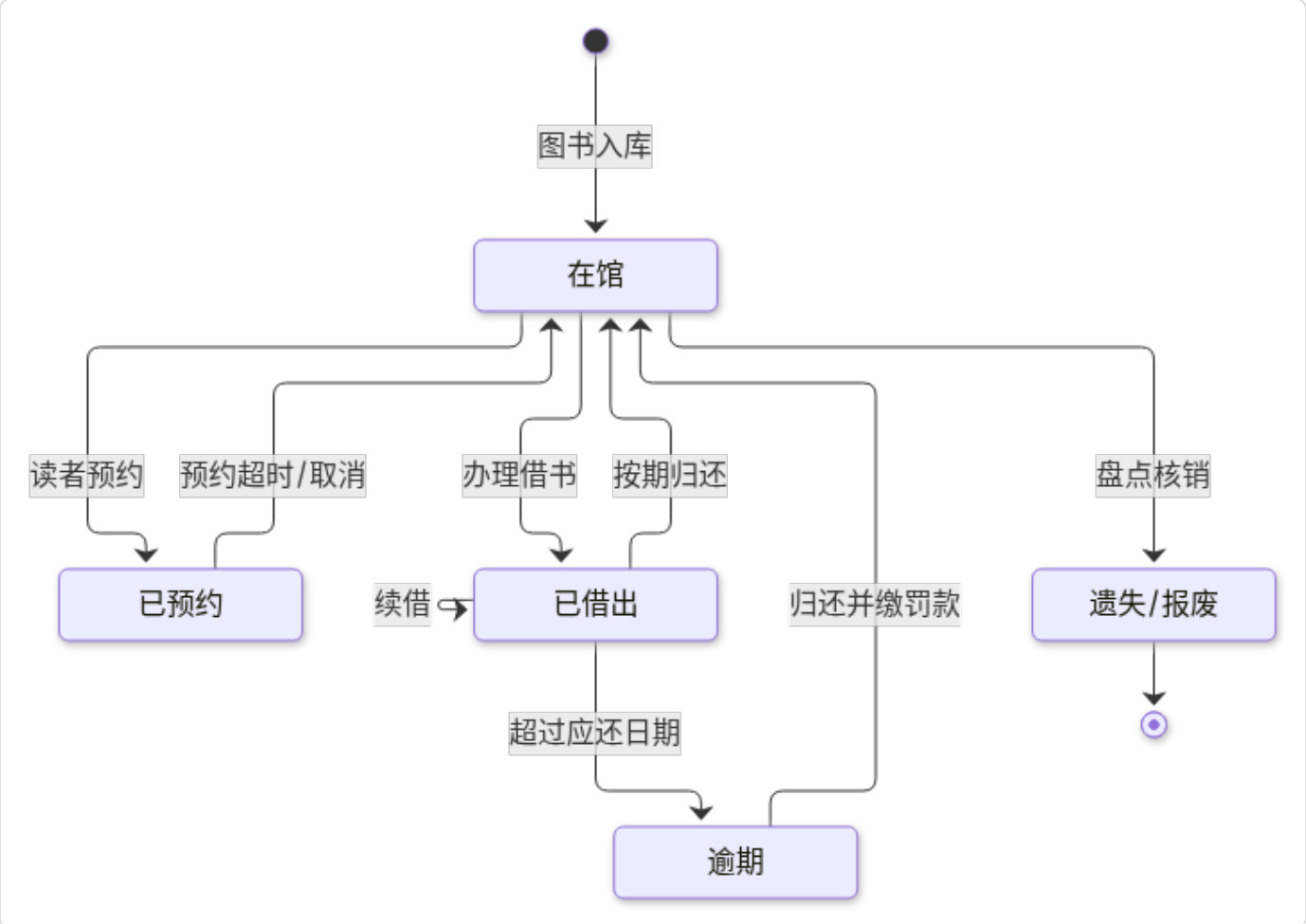
6. 部署图（Deployment Diagram）




```
graph TB
    subgraph 用户终端
        PC[PC浏览器]
        Phone[手机/微信小程序]
    end
```

```
    subgraph cloud[应用服务器节点 : Linux + Nginx]
        NG[Nginx 反向代理]
        APP[图书管理系统Web 应用 / REST API]
    end
    subgraph dbs[数据库服务器节点 : Linux + MySQL]
        MYSQL[(MySQL 主从)]
    end
    PC -->|HTTPS| NG
    Phone -->|HTTPS / API| NG
    NG --> APP
    APP -->|JDBC / TCP 3306| MYSQL
```

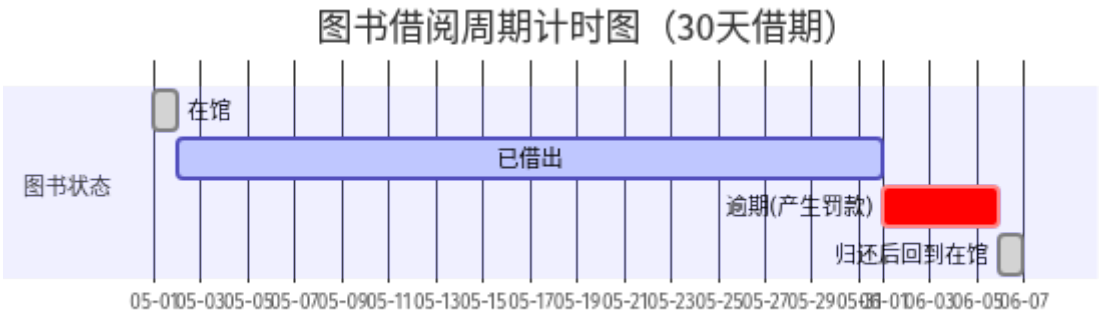
7. 状态图（State Diagram——图书对象的生命周期）




```
stateDiagram-v2
    [*] --> 在馆 : 图书入库
  在馆 --> 已预约 : 读者预约
  已预约 --> 在馆 : 预约超时/取消
  在馆 --> 已借出 : 办理借书
  已借出 --> 已借出 : 续借
  已借出 --> 逾期 : 超过应还日期
  逾期 --> 在馆 : 归还并缴罚款
  已借出 --> 在馆 : 按期归还
```

```
在馆 --> 遗失/报废 : 盘点核销
遗失/报废 --> [*]
```

8. 计时图（Timing Diagram——图书从借出到归还）




```
gantt
    title 图书借阅周期计时图（30天借期）
    dateFormat  YYYY-MM-DD
    axisFormat  %m-%d
    section 图书状态
  在馆              :done, s1, 2026-05-01, 1d
  已借出            :active, s2, 2026-05-02, 30d
  逾期(产生罚款)  :crit,  s3, 2026-06-01, 5d
  归还后回到在馆    :done,  s4, 2026-06-06, 1d
```

> 计时说明：0~1 天处于「在馆」；第 1~30 天为合法借期「已借出」；第 31 天起进入「逾期」状态并按天累计罚款；第 36 天归还后状态回到「在馆」。

三、实验收获

通过本次 UML 建模实验，我对软件工程中可视化建模的思想有了更深入的理解，主要收获如下：

1. **加深了对 UML 各类图适用场景的理解。** 通过为图书管理系统绘制 8 种不同的 UML 图，我体会到每类图都有其侧重的视角：用例图从用户需求出发描述系统功能，类图和对象图刻画系统的静态结构，活动图和状态图表达动态行为，组件图和部署图反映系统的物理实现结构，计时图则补充了对时间约束的描述。只有把静态与动态、逻辑与物理结合起来，才能完整地刻画一个系统。

2. **掌握了事件流的写法与用例建模的关系。** 在分析茶颜悦色小程序「修改用户名 / 密码」功能时，我学会了区分基本事件流与备选事件流（如密码错误、格式不合法、网络异常等分支），并体会到事件流是绘制时序图和用例图的基础——先写清楚"谁在什么时候做什么、出错怎么办"，图形建模才不会遗漏关键分支。

3. **提高了从需求抽象出模型的能力。** 例如在类图设计中，通过抽取 User 抽象父类来复用 Reader 与 Admin 的公共属性，用 BorrowRecord 关联类来表达读者与图书之间多对多的借阅关系，我认识到好的模型不仅要"画得像"，更要结构合理、便于扩展。

4. **学会了使用建模工具。** 本次实验我使用 Mermaid 以文本方式编写图形，感受到"代码即图"的优势：模型可以纳入版本管理、修改方便、可重复生成，同时也锻炼了自己阅读和编写规范化描述文档的能力。

5. **认识到建模对团队协作的意义。** 统一的图形语言让开发、测试与客户之间有了共同的沟通载体，能在编码之前尽早发现需求理解上的偏差，降低后期返工成本。这让我理解了"先建模、后编码"这一软件工程基本思想的实际价值。

总之，本次实验让我完成了从"会画图"到"会建模"的思维转变，为后续的软件开发课程和项目实践打下了良好基础。